Table of contents

1. Overview

Retrospective is a software utility and graphical user interface to reconstruct self-gated cardiac or respiratory CINE MRI acquired with an MR Solutions preclinical MRI system.

Reconstruction is based on compressed-sensing after retrospective sorting of the data into cardiac and/or respiratory time frames, using a navigator signal for cardiac and respiratory synchronization.

2. Contact

Retrospective and this manual is written by:

Gustav Strijkers
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
g.j.strijkers@amsterdamumc.nl

3. Disclaimer

The software has been extensively tested using mouse data acquired with an MR Solutions preclinical 7.0T/24cm system.

However, this does not warrant the functions contained in the app will meet your requirements or that the operation of the app will be uninterrupted or error-free.

In case of questions or issues, please contact Gustav Strijkers.

4. Citing the use of Retrospective

Retrospective is free of use. However, proper referencing the use of Retrospective in scientific publications is highly appreciated.

Citations to Retrospective can be done using the following reference (**link**):

Daal, M.R.R., Strijkers, G.J., Calcagno, C., Garipov, R.R., Wüst, R.C.I., Hautemann, D., Coolen, B.F. *Quantification of Mouse Heart Left Ventricular Function, Myocardial Strain, and Hemodynamic Forces by Cardiovascular Magnetic Resonance Imaging*. J. Vis. Exp. (171), e62595, doi:10.3791/62595 (2021).

5. Installation notes

Software download

Matlab source code and a Windows standalone version (using the free Matlab runtime engine) can be downloaded from github:

`https://github.com/Moby1971?tab=repositories`

Installation of the Windows standalone version

`Retrospective_app_installer.exe`

Will install the Matlab runtime engine and the Retrospective app.

Bart toolbox download

The advanced reconstruction options in Retrospective require the Bart reconstruction toolbox, which can be downloaded from:

`https://mrirecon.github.io/bart/`

Bart toolbox installation in OSX with MacPorts

(1) Install the Xcode command line tools

```
$ xcode-select --install
```

(2) Install MacPorts

From http://www.macports.org/. It is recommended to install a newer version of gcc from MacPorts, which step 3 does.

(3) Install packages

```
$ sudo port install fftw-3-single
$ sudo port install gcc14
$ sudo port install libpng
$ sudo port install openblas
$ sudo port install flock
$ sudo port install gmake
```

(4) Download and unpack Bart

```
$ wget https://github.com/mrirecon/bart/archive/v0.9.00.tar.gz
$ tar xvfz v0.9.00.tar.gz
$ cd bart-0.9.00
```

(5) Build and install

```
$ CC=gcc-mp-14 gmake
$ sudo gmake install
```

The compiler name must match the gcc version installed in step 3:
gcc14 gives gcc-mp-14, gcc15 gives gcc-mp-15.

(6) Check that it works

```
$ bart version
```

Bart toolbox installation in OSX with HomeBrew

(1) Install the Xcode command line tools

```
$ xcode-select --install
```

(2) Install Homebrew

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

(3) Install the packages Bart needs

```
$ brew install gcc
$ brew install libpng
$ brew install fftw
$ brew install openblas
$ brew install make
$ brew install llvm libomp
```

The Homebrew package that provides GNU make is called make.
It installs the program under the name gmake.

(4) Download and unpack Bart

```
$ wget https://github.com/mrirecon/bart/archive/v0.9.00.tar.gz
$ tar xvfz v0.9.00.tar.gz
$ cd bart-0.9.00
```

(5) Build and install

```
$ CC=gcc-15 MACPORTS=0 make
$ sudo make install
```

The compiler name must match the version Homebrew installed.
Check it with `ls /opt/homebrew/bin/gcc-*`

(6) Check that it works

```
$ bart version
```

Bart toolbox installation in Windows 11

On Windows, Bart runs inside the Windows Subsystem for Linux (WSL).

(1) Install WSL.

Start a Windows PowerShell as administrator and run:

```
wsl --install
```

This enables the feature, installs WSL 2 and installs Ubuntu in one step. A restart is needed. The first time you start the Linux prompt you will be asked to create a user account.

On older systems, or if wsl --install is not recognized, use the manual route instead:

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux
```

Then restart, list the available distributions with `wsl -l -o`, install one with `wsl --install -d <distribution name>`, and upgrade to WSL 2.
See https://pureinfotech.com/install-wsl-windows-11/ for details.

(2) Update the Linux distribution

Everything from here on is typed at the Linux prompt, not in PowerShell.

```
$ sudo apt-get update
$ sudo apt-get dist-upgrade
```

(3) Install the packages Bart needs

```
$ sudo apt-get install make gcc libfftw3-dev liblapacke-dev libpng-dev
libopenblas-dev gfortran
```

(4) Download and unpack Bart

```
$ wget https://github.com/mrirecon/bart/archive/v0.9.00.tar.gz
$ tar xvfz v0.9.00.tar.gz
$ cd bart-0.9.00
```

(5) Build and install

```
$ make
$ sudo make install
```

If you want GPU support, do not run this yet — follow the next section instead, which builds Bart with CUDA. GPU support is highly recommended.

(6) Check that it works

At the Linux prompt:

```
$ bart version
```

And from Windows, which is how Retrospective calls it:

```
wsl bart version
```

Both should print a version number.

Building Bart with GPU support on Windows (highly recommended)

The reconstruction is considerably faster on a GPU.

(1) Install the NVIDIA driver in Windows

Follow https://docs.nvidia.com/cuda/wsl-user-guide/index.html. The Windows driver is what provides GPU access inside WSL — do not install a Linux display driver inside WSL.

(2) Install the CUDA toolkit inside WSL

Get the current installer from https://developer.nvidia.com/cuda-downloads, choosing Linux → x86_64 → WSL-Ubuntu.

For CUDA 12.8 this is:

```
$ wget https://developer.nvidia.com/compute/cuda/12.8.0/local_installers/
  cuda_12.8.0_570.86.10_linux.run
$ sudo sh cuda_12.8.0_570.86.10_linux.run
```

(3) Find the compute capability of your card

Bart has to be compiled for the GPU generation it will run on. A binary built for the wrong generation compiles without complaint and then fails at the first calculation.

Look your card up at https://developer.nvidia.com/cuda-gpus and note its compute capability:

GPU generation	Example cards	Value to use
Pascal	GTX 1060, 1080, Titan X	sm_61
Turing	RTX 2060 – 2080	sm_75
Ampere	RTX 3060 – 3090, A100	sm_80 / sm_86
Ada Lovelace	RTX 4060 – 4090	sm_89
Blackwell	RTX 5070 – 5090	sm_120

This is the same kind of problem as the PyTorch installation described later in this section: which build is correct depends on the card in the machine, and no single value suits every computer.

(4) Set the environment variables

Set these in the same shell you will build in. Adjust `CUDA_BASE` if you installed a different CUDA version, and `GPUARCH_FLAGS` to the value from step 3.

```
$ export CUDA=1
$ export CUDA_BASE=/usr/local/cuda-12.8
$ export CUDA_LIB=lib64
$ export GPUARCH_FLAGS="-arch=sm_61"
$ export PATH=$CUDA_BASE/bin:$PATH
$ export LD_LIBRARY_PATH=$CUDA_BASE/$CUDA_LIB:$LD_LIBRARY_PATH
```

The first four are the variables Bart's Makefile reads. PATH makes `nvcc` reachable, and `LD_LIBRARY_PATH` lets the finished program find the CUDA libraries when it runs.

(5) Build

```
$ make -j$(nproc) CUDA=1 CUDA_BASE=$CUDA_BASE CUDA_LIB=$CUDA_LIB
  GPUARCH_FLAGS="$GPUARCH_FLAGS"
$ sudo make install
```

If the build fails at the linking stage with an error about `libstdc++` or `libgcc`, point the linker at the installed gcc explicitly and build again.
Check which version you have with `ls /usr/lib/gcc/x86_64-linux-gnu/` and use that number:

```
$ export LDFLAGS="-L/usr/lib/gcc/x86_64-linux-gnu/13 $LDFLAGS"
```

(6) Test the GPU build

```
$ make utest_gpu
```

How Retrospective finds Bart

In the normal case there is nothing to configure: sudo make install puts Bart in `/usr/local/bin`, which is where the app looks.

MacOS

Retrospective reads the environment variable `TOOLBOX_PATH`. When it is not set, it falls back to `/usr/local/bin` and then to `/usr/bin`.

If Bart is installed somewhere else, set `TOOLBOX_PATH` to the folder that contains the bart program. Setting it in your shell profile is not enough — Matlab started from the Dock does not read your shell startup files, so it will not see it. Either start Matlab from a terminal, or set it inside Matlab before starting Retrospective:

```
>> setenv('TOOLBOX_PATH','/usr/local/bin')
```

Windows

Retrospective runs Bart inside WSL, as `wsl bart`. What matters is that bart is on the PATH inside your Linux distribution; TOOLBOX_PATH is not used on Windows. `sudo make install` takes care of this.

Check from a Windows command prompt with:

```
wsl bart version
```

When Retrospective starts it reports the Bart version it found in the message box. If Bart was not found, the message box says so and the app continues with its own Matlab reconstruction.

Installing Python for deep learning reconstruction and LV segmentation

Two features use Python: the deep learning (DRIM) reconstruction and the LV segmentation. Both are optional; the rest of the app runs without them.

You do not have to install the Python packages yourself. The first time a deep learning feature is used, Retrospective creates its own Python environment and installs what it needs into it. This takes a few minutes and happens only once. Progress is written to the message box, and a full log is kept in `retroPythonSetup.log` in the temporary folder.

The environment is created in your home directory:

```
macOS      ~/venv_retro
Windows    $HOME/venv_retro   (inside WSL, not on the Windows drive)
```

The following packages are installed, with fixed versions so that every machine ends up with the same thing:

```
lightning       2.6.5
numpy           2.3.5
scipy           1.15.3
PyYAML          6.0.3
onnxruntime     1.28.0
```

PyTorch is deliberately not in that list, and is handled separately, because which build is correct depends on the machine rather than on a version number. See "Graphics card support" below.

What you need to provide yourself !

On most computers, nothing.

Retrospective looks for a Python it can use, and if it does not find one it installs a Python of its own. Three routes are tried in turn: Conda, if the machine already has it; otherwise a self-contained Python that is downloaded and checked against its published checksum before anything in it is run; otherwise Homebrew, on macOS. None of them needs administrator rights, and none of them disturbs a Python you already use — the one the program installs for itself goes in a folder of its own:

```
macOS      ~/.retro-python
Windows    $HOME/.retro-python   (inside WSL)
```

and nothing else on the machine is touched. You are asked before this happens, and the dialog says where the interpreter will go.

Which Python versions can be used ?

```
Python 3.11, 3.12 or 3.13.
```

The pinned packages install on those three and no others: below them `numpy` and `onnxruntime` publish nothing that fits, above them `scipy` does not. A Python outside that range is not an error and nothing goes wrong — the program simply does not build on it and uses one of its own instead. It is worth knowing on Windows, where a stock Ubuntu in WSL comes with Python 3.10 and will therefore be passed over. When the program installs a Python for itself it installs 3.12.

PyTorch

If PyTorch is already installed and working on your computer, Retrospective uses it as it is. This is the better case, because a PyTorch that already runs on your hardware is by definition the right build for it. If no PyTorch can be found at all, the program installs one itself.

MacOS

Installing Python together with PyTorch — through Miniconda or Homebrew — gives the best result, because PyTorch then comes from a source that knows your hardware. It is not required. It does not need to be the Python that Matlab finds by default; the program locates a suitable interpreter by itself, and installs one if there is none.

Windows

Python runs inside WSL. See the Bart installation section above for setting up WSL. If WSL is not installed at all, the message box says so and gives you the command. That is the one step in the chain the program cannot take for you, because `wsl --install` needs administrator rights and a restart. The rest of the app is unaffected and works normally; the deep learning reconstruction and the LV segmentation are simply switched off.

Inside WSL there is nothing else to prepare. On Ubuntu the module that creates virtual environments is packaged separately, and if it is missing the program works around it — first with `virtualenv`, and failing that by building on a Python of its own. If you would rather give it the short route:

```
$ sudo apt install python3-venv
```

but this is not required, and the program only suggests it in the message box if every other route has failed as well.

Graphics card support

The deep learning features are considerably faster on a graphics card, and on a computer without one the reconstruction may be too slow to be practical.

You do not have to choose the right packages yourself. Every time it starts, Retrospective checks what graphics hardware is present and whether the installed PyTorch and `onnxruntime` can actually use it. If they cannot, it offers to replace them with versions that can, and installs them for you. This is checked at every start, not only when the environment is first built, so a graphics card or driver installed later is picked up the next time you start the program — you do not need to reinstall anything by hand.

What this means in practice:

Computer	Reconstruction uses	Segmentation uses
Apple silicon Mac	the built-in GPU (Metal)	the built-in GPU (CoreML)
NVIDIA graphics card	the card (CUDA)	the card (CUDA)
No graphics card	the processor	the processor

On a computer with an NVIDIA card, the CUDA driver has to be installed and working first — on Windows this means the driver in Windows, with GPU access enabled for WSL, as described in the Bart section above. Retrospective can install the right Python packages, but it cannot install your graphics driver.

If the replacement is offered and you decline, everything still works on the processor, and you will be asked again the next time you start the program.

Checking what was actually used. Retrospective reports the device at the start of every reconstruction, in the message box and in the log file `retroAIdrim.log`, as a line like:

```
GPU available: True (mps), used: True
```

If this says no GPU is available on a computer that has one, the graphics driver is not reachable — on Windows, check that `wsl nvidia-smi` lists your card.

6. Running the software

Running in Matlab 2025a

The Retrospective software can be started from its root directory from the command line.

```
>> retrospective10
```

Additional licenses may be required.

```
MATLAB
Image Processing Toolbox
Parallel Computing Toolbox
Signal Processing Toolbox
```

Running the Windows standalone

The Windows standalone version can be run from the start menu or the desktop icon.

7. Basic operation

The Retrospective app operates from a single window with 5 main panels.

Panel 1: Dashboard

This panel contains the task buttons that control the reconstruction process. The dashboard tasks need to be completed from top to bottom. A green light next to the task indicates that a task has been completed. When the light is yellow the task is busy. Red indicates not completed yet.

Panel 2: Navigator / Heart / Respiration / Settings

This panel shows the raw navigator signal, the navigator frequency spectrum, and some advanced navigator analysis parameters.

Panel 3: Navigator analysis

This panel shows the navigator signals filtered for cardiac and respiratory motion, as well as the heart and respiratory rates as function of time.

Panel 4: Sequence / Geometry / Reco Parameters / Radial

This panel lists the relevant acquisition and reconstruction parameters.

Panel 5: Reco / L-curve / Radial

In this panel the reconstructed CINE movie is shown. There are advanced sub-panels for optimization of compressed-sensing reconstruction (L-curve analysis) and Radial reconstructions.

Step 1: Loading data

Press Load MRD to import load the MRD raw data file *.

The data should include navigator data for cardiac and respiratory synchronization.

For data acquired with multiple receivers (phased-array RF coil) select 1 of the MRD files.

Relevant acquisition parameters will be shown in panel 2, including the k-space trajectory (linear, zigzag, or user defined).

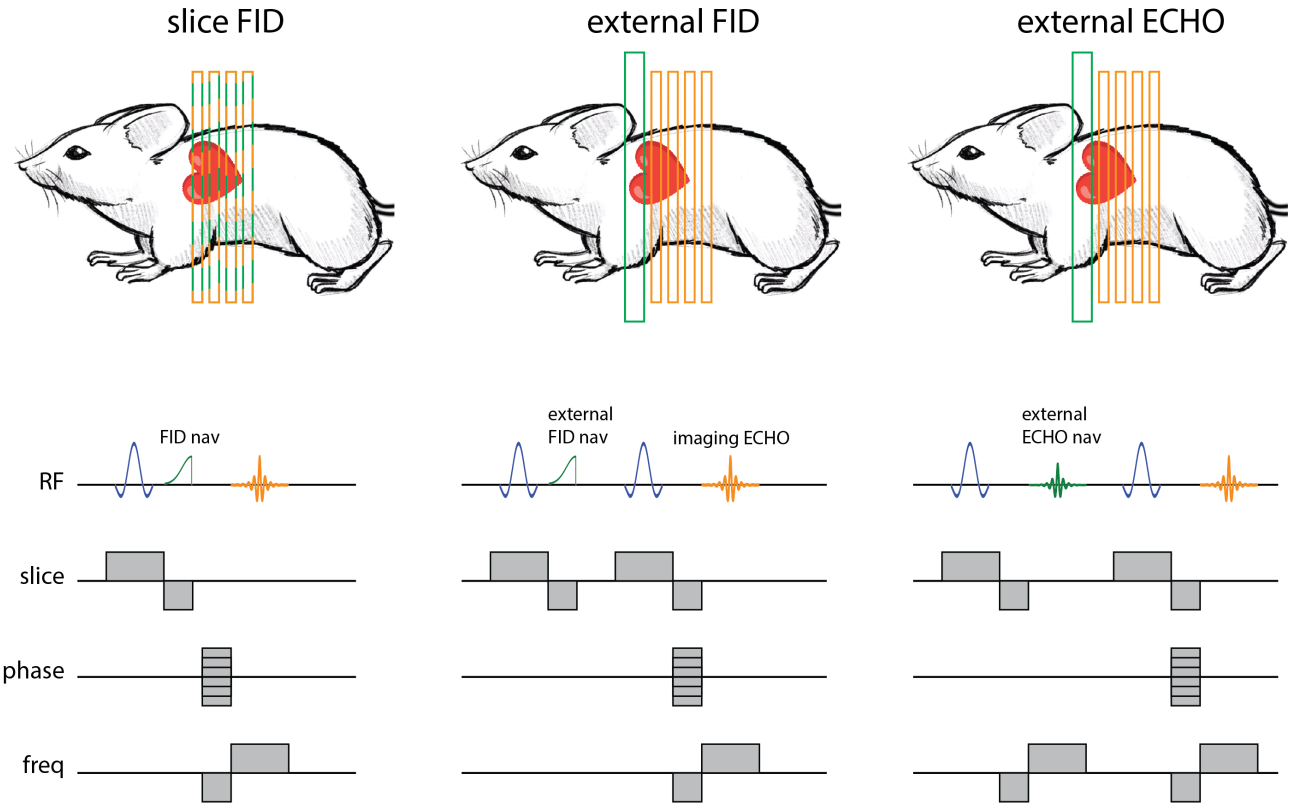
Step 2: The navigator signal

There are **3** types of navigators, which can be acquired independently and in combination, and in both 2D and 3D imaging.

- **Slice FID navigator** (left): This navigator is the free-induction-decay (green) generated from the rewinder of the slice selection gradient.

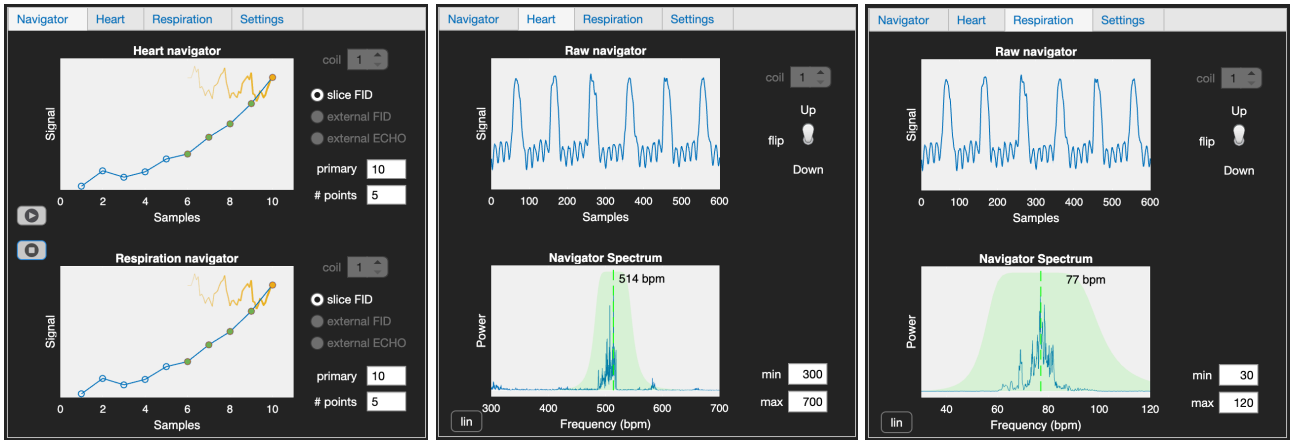
- **External FID navigator** (middle): The free-induction-decay (green) produced by the rewinder of a slice selection gradient in a separate non-spatially encoded slice.

- **External ECHO navigator** (right): The gradient-echo (green) produced in a separate slice with a readout gradient in a single dimension. The resulting navigator is the Fourier transform of the echo and therefore a 1-dimensional image (readout profile) of the object.



* since version 7.3.1 it is also possible to load fid.orig and rawdata.job0 Cartesian data

Depending on which type of navigator is available, one can choose slice FID, external FID, or external ECHO to extract heart and respiratory motion in the *Navigator* tab. The resulting filtered signals for heart and respiratory motion can be inspected in the *Heart* and *Respiration* tabs.

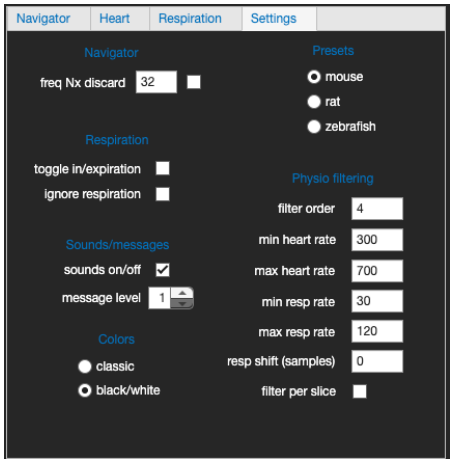


For multi receiver coil data, individual navigators can be inspected with the **coil** toggle switch.

The respiration peaks in the navigator should be positive; in case they are negative, flip the signal with the **Flip** switch.

If cardiac and respiratory oscillations are not clearly visible, change the **primary** point slightly and/or increase **# points** (typically **# points** = 1 to 5) in the Settings tab to see whether the quality improves.

When auto reco ☑ is enabled, retrospective will watch the MRD datafile directory and start automatic reconstruction when a new MRD file is added for seamless integration with the MR Solutions preclinical imaging software.



The *Settings* tab contains 3 presets for navigator filtering of mouse, rat, and zebrafish data. These adapt the values **min/max heart** and **respiration rates**, which determine the search windows for the heart and respiration rate.

The parameter **freq Nx discard** determines the number of samples that are discarded between the end of the navigator and the beginning of the imaging echo. It is active when checked. If an FID artifact is visible in the reconstructed images/movie, increase this number. This parameter is usually only relevant for FLASH sequence version < 710.

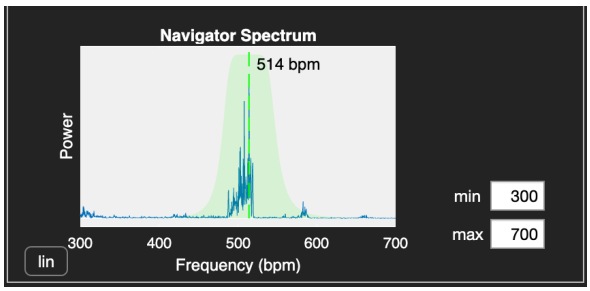
Images can be reconstructed during in- or expiration. Default is reconstruction during expiration, in the intervals between breath-takes. For reconstruction of images during inspiration, tick the *toggle in/expiration* check box.

Discarding k-lines during respiration can be overruled by ticking the *ignore respiration* checkbox.

Error and warning sounds can be switched on or off with the *sounds on/off* checkbox.

Check the corresponding box to switch between *black/white* and *classic* user interface.

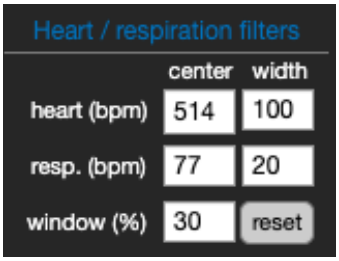
Step 3: Inspecting the navigator power spectrum



The bottom parts of the *Heart* and *Respiration* tabs show the power frequency spectrum of the raw navigator signal.

Typically, the spectrum looks as the figure above, with peaks (and higher harmonics) corresponding to the respiratory and cardiac frequencies. The app automatically determines the most likely heart and respiration frequencies.

The green shaded area indicates the position and width of the Butterworth filters, which are used to filter the raw navigator and extract the periodic cardiac and respiratory motion as function of time.



In case the automatic respiration and/or cardiac frequency detection fails, the **center** and **width** filter values can be adjusted in the heart/respiration filters panel, shown below.

Center and **width** determine the center-frequency and width of the Butterworth filters for navigator filtering.

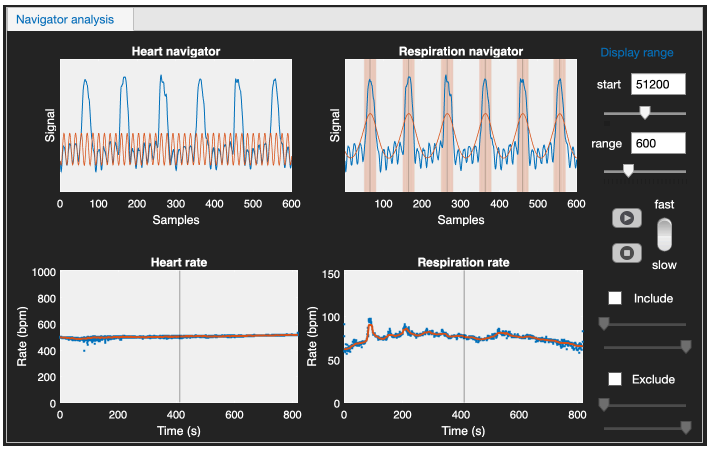
Window (%) determines the percentage of the data that is discarded during respiratory motion.

Press reset to reset the values.

Step 4: Filter the navigator

Press Filter to filter the raw navigator signal and extract cardiac and respiratory motion as function of time.

The result may look like this:



The sliders can be used to change the **display range** to inspect the full data set.

Use 

buttons to automatically advance the navigator and inspect the data.

The top left figure shows in blue the raw navigator signal and in red the signal filtered for cardiac motion. The gray vertical lines indicate the peaks in the oscillations, which can be used to check the alignment between the red and blue signals.

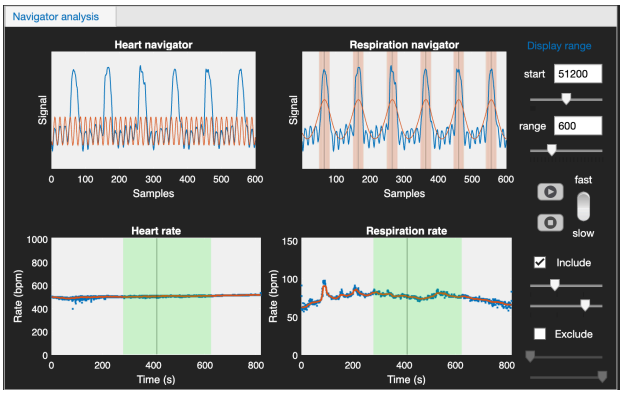
Bottom left is the resulting heart rate as function of during the full acquisition. The blue line is the heartbeat to heartbeat variation, whereas the red line is a smoothed trend-line.

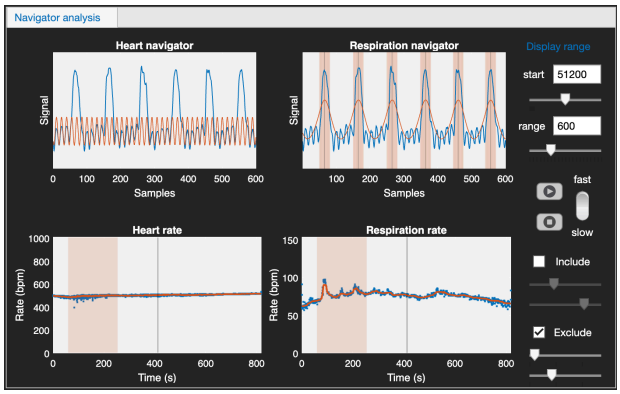
The top right figure shows in blue the raw navigator signal and in red the signal filtered for respiratory motion. The gray vertical lines indicate the peaks in the oscillations, which can be used to check the alignment between the red and blue signals. The shaded red areas indicates the part of the data which is discarded during respiratory motion.

Bottom right is the resulting respiration rate as function of during the full acquisition. The blue line is the respiration to respiration variation, the red line is a smoothed trend-line.

Heart and respiration rates should be smoothly varying over time. If this is not the case, adjust the Butterworth filter settings.

Including / excluding data

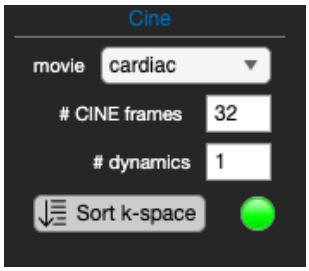




In case of irregular respiration or heartbeat part of the data can be included (green) and/or excluded (red) by checking the **Include** or **Exclude** boxes or both. Including or excluding data is not possible for multi-slice 2D data.

After this, press Filter again to filter the data with the new settings.

Step 5: Sort the k-space



Choose the desired sorting method, number of frames, and number of dynamics. In the presets panel you can select some pre-defined sorting options.

Sorting methods

(1) **Cardiac**

Cardiac phase-binning. Each heartbeat is divided into an equal number of cardiac CINE frames. The average time between cardiac frames is determined from the average heart-rate and the number of cardiac frames. This option is preferred for most applications, particularly for high frame-rate reconstructions for analysis of diastolic function.

(2) **Cardiac abs.**

Cardiac absolute-binning. A fixed frame time is predetermined from the average heart-rate and the number of cardiac CINE frames. This option could be more accurate for reconstruction of systolic function in the presence of heart-rate variations, but leads to temporal blurring in the diastolic phase.

(3) **Respiratory**

Respiratory phase-binning. Each respiration cycle is divided into an equal number of respiratory frames. The average time between respiratory frames is determined from the average respiration-rate and the number of desired respiration frames. When using this option, **window (%)** is neglected.

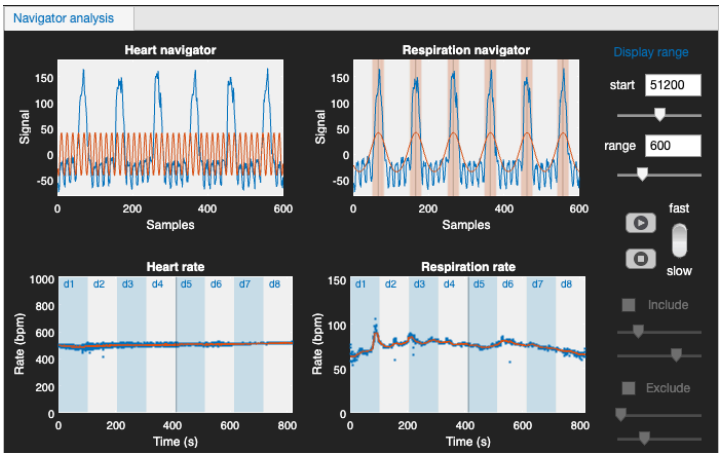
(4) **Realtime**

With the realtime option selected the data is sorted into 1 CINE movie per heart beat for fast dynamic imaging, for example during injection of a contrast agent. This option requires a short-as-possible TR and a suitable sparse k-space trajectory that results in sufficient k-lines per frame for compressed-sensing and/or parallel imaging image reconstruction. The number of dynamics will be automatically set to the total number of heartbeats in the acquisition and respiratory motion is disregarded. The parameter sharing in the advanced settings panel determines the number of k-lines that are shared from neighboring dynamics in order to increase k-space filling and image quality at the expense of temporal averaging.

Press Sort k-space to start sorting the data into k-space.

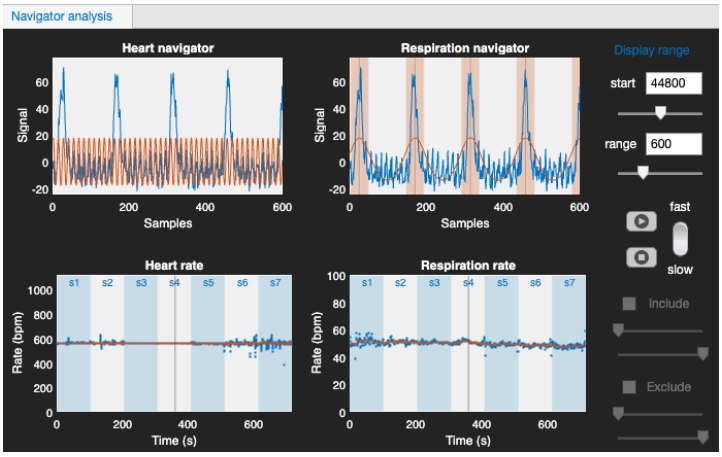
Dynamics

It is possible to reconstruct data in a dynamic time series, for example for dynamic-contrast enhanced acquisitions. Choose the desired number of dynamics, and continue with sorting k-space and reconstruction. The maximum number of dynamics is set to 600, but remember that k-space will become very sparse with high number of dynamics and reconstruction might take very long. The dynamic time-frames will be indicated in the heart rate and respiratory rate plots in blue/white shaded areas as shown below.



Multi-slice 2D data

For multi-slice 2D acquisitions, the heart rate and respiration rate plots show when the subsequent slices were acquired with blue/white shaded areas. For multi-slice 2D data it is not possible to reconstruct different dynamics nor is it possible to include or exclude part of the scan.

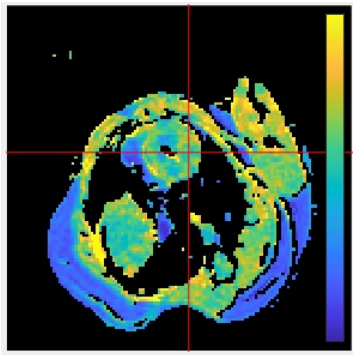


3D data

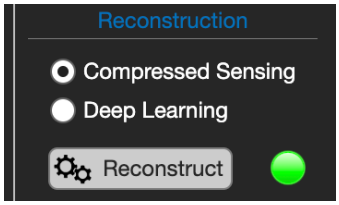
Version 9 of Retrospective supports reconstructions of 3D CINE data.

Variable flip-angle data for T1-mapping

Cardiac quantitative T1 maps can be acquired using the variable flip angle method (see: Coolen et al. NMR Biomed 24, 154–162 (2011)). This option requires a 3D self-gated T1-weighted (RF and gradient spoiled) acquisition with variable flip angle (typically 5 angles, for example 2°, 5°, 10°, 15°, 20°). The varying flip angles should be acquired in a single acquisition in which the flip angles are evenly distributed over the k-space repetitions. After reconstruction data should be exported as a new MRD file for T1 mapping using the **T1mappD** application available on GitHub.



Step 6: Reconstruction of the data



Before reconstruction, select the appropriate reconstruction settings and regularization parameters in the **Reco Parameters** panel.

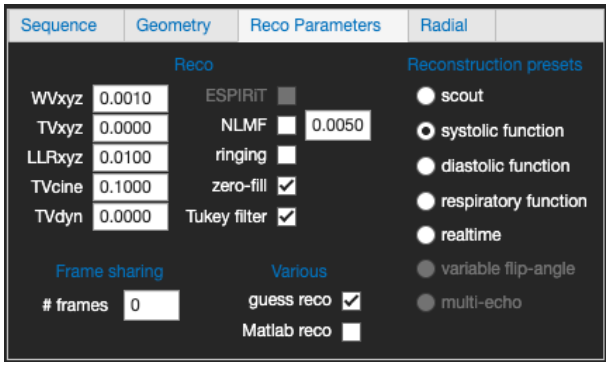
Two reconstruction methods are available, selected in the Reconstruction panel of the dashboard:

(A) Compressed Sensing — the standard method, described below.

(B) Deep Learning — a trained neural network, described further down.

Press  to start the reconstruction.

A. Compressed Sensing reco parameters



Some parameters are not available without the BART toolbox.

(1) **WVxyz**

Wavelet regularization parameter in the spatial (x, y, and z) dimensions. For the mouse cardiac CINE, a low (0.001) or zero value resulted in the best movie quality.

(4) **TVxyz**

Total variation constraint in the spatial dimension. Higher values smooth the movie in the spatial domain. For mouse CINEs, this parameter does not seem to improve image quality. Recommended value therefore is zero.

(2) **LLRxyz**

Locally Low Rank regularization parameter.

(3) **TVcine**

Total variation constraint in the CINE dimension. Higher values lead to improved image quality but increased temporal smoothing. For the mouse CINEs, a value of 0.1 seems a good compromise between image quality and temporal smoothing.

(5) **TVdyn**

Total variation constraint in the dynamics dimension.

(6) **ESPIRiT**

ESPIRiT reconstruction (Magn Reson Med. 2014 Mar 71(3): 990–1001.). This includes GRAPPA and SENSE reconstruction.

B. Deep learning reconstruction

Select Deep Learning in the Reconstruction panel. The deep learning reconstruction uses DRIM, a Dynamic Recurrent Inference Machine trained on rodent cardiac CINE data. Rather than solving an optimization problem as compressed sensing does, the network starts from the undersampled images and improves them in a fixed number of refinement steps, using the measured k-space at every step to keep the result consistent with the data that was actually acquired.

Requirements

The Python environment described in section 5.
macOS, or Windows with WSL.

The app chooses the trained model automatically from the amount of undersampling, so there is nothing to set. The k-space filling percentage shown in the Sequence tab determines the choice: above about 67 % filling the model trained on lightly undersampled data is used, below that the model trained on more strongly undersampled data.

Points to be aware of

The images are zero-filled to a 256 x 256 matrix, which is the matrix the network was trained on. The reconstruction runs as a background process. No terminal window appears. The progress bar advances as the network iterates, and messages appear in the message box.

The network is applied once for every receive coil and every dynamic, so multi-coil or multi-dynamic data takes proportionally longer.

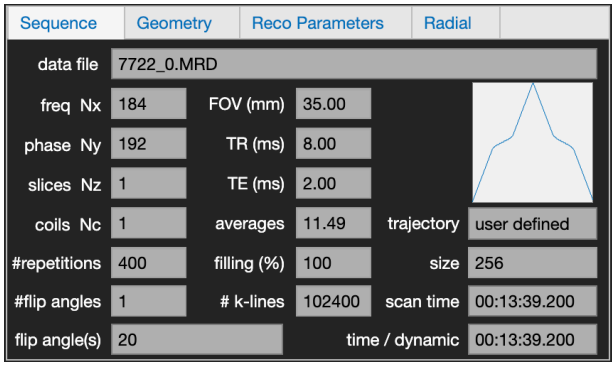
Memory use and reconstruction time grow with the number of CINE frames and the number of slices. On a machine with limited memory, reconstructing many frames at once can be slow or stall your computer.

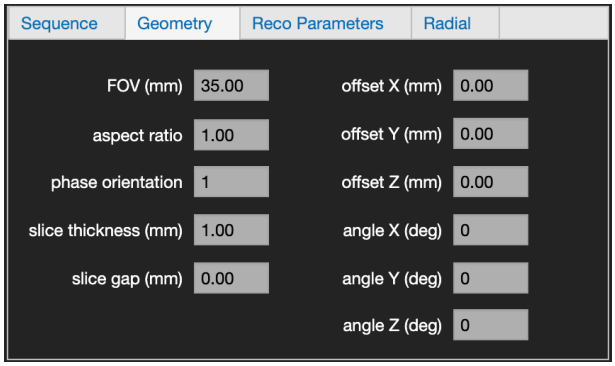
The reconstruction parameters in the Reco Parameters panel apply to compressed sensing and have no effect on the deep learning reconstruction.

If the Python environment is not available, the deep learning option cannot be selected and the message box explains what is missing.

Sequence / Geometry

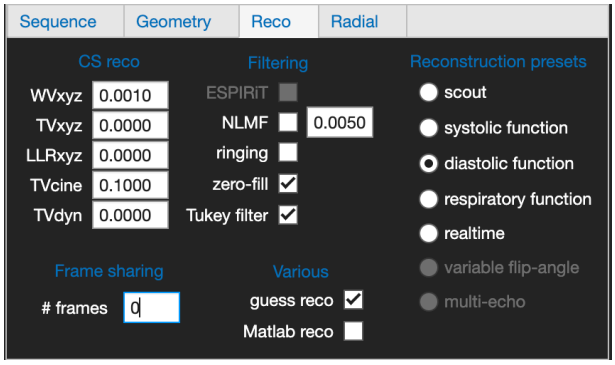
The Sequence and Geometry tabs contain information on the sequence parameters and the image geometry.





Reco Parameters

The Reco Parameters tab contains additional reconstruction settings.



Frame sharing: Binning of cardiac and/or respiratory motion states can be done using a Gaussian soft weighting of k-lines from neighboring frames (see: Küstner et al., NMR Biomed e4409 (2020). doi:10.1002/nbm.4409) to increase k-space filling and SNR. The #frames value determines the number of shared frames.

Reconstruction presets: Provides some pre-determined reconstruction settings for different applications.

NLMF: Applies a non-linear means filter to the images with set weight.

ringing: Checkbox to switch a anti-ringing filter on or off.

zero-fill: Checkbox to switch zero-filling on or off.

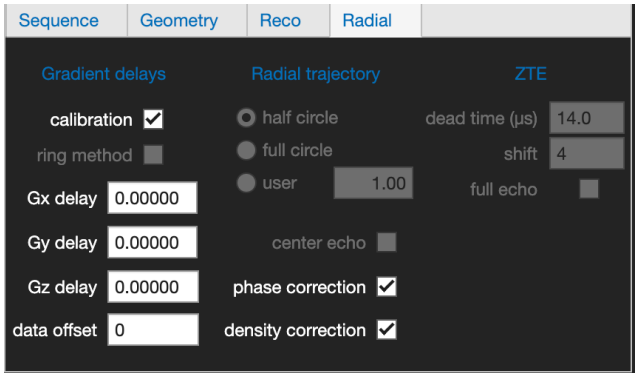
Tukey filter: Checkbox to switch ellipsoid Tukey filtering of k-space on or off.

guess reco: Check the checkbox for the application to make an educated guess what type of reconstruction is appropriate. This is especially useful when doing automatic reconstruction.

Matlab reco: One can force a Matlab-based reconstruction by checking the checkbox.

Radial

The radial tab contains additional settings for radial and UTE reconstructions.



Tick the *calibration* checkbox for automatic adjustment of the gradient delays. When the *ring method* checkbox is ticked, the RING method described by Rosenzweig et al. is used [Rosenzweig, S., Holme, H. C. M. & Uecker, M. Simple auto-calibrated gradient delay estimation from few spokes using Radial Intersections (RING). Magn Reson Med 81, 1898–1906 (2019)]. When *ring method* is unchecked, a recursive image-based gradient delay optimization is performed. The latter method is much slower.

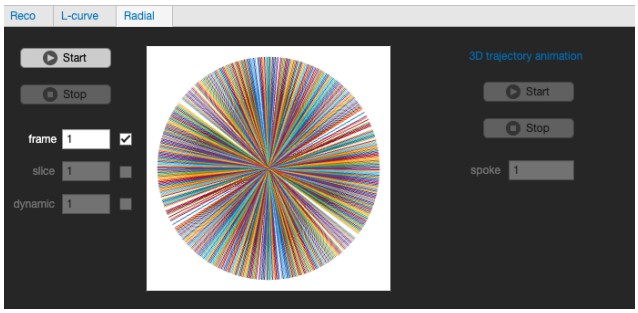
Currently three 2D radial trajectories are currently supported: half-circle (0:1:180 degrees), full-circle (0:1:360 degrees), and user defined (reading from MRD file, work in progress).

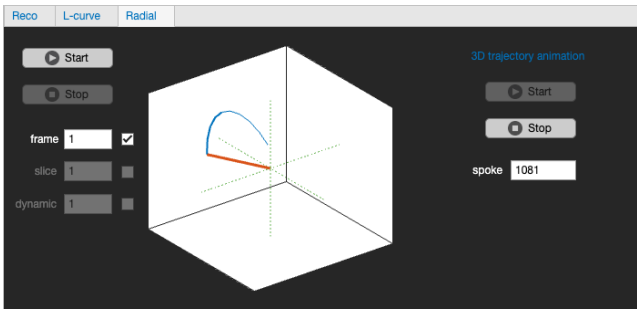
With *center echo* checked, the echoes are centered before reconstruction.

Phase correction performs a phase correction of the echoes before reconstruction.

Density correction performs a 2D or 3D density correction in k-space to account for non-uniform sampling. This setting only applies to recontructions with *Bart*.

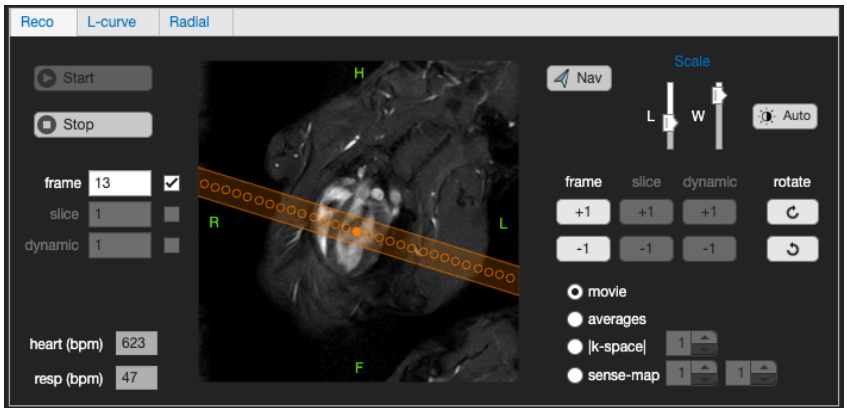
In the **Radial** tab in panel 5 one can view the 2D or 3D radial k-space filling and animate the 3D radial trajectory





Step 7: Viewing the movie

The resulting CINE MRI can be viewed in the Movie panel. Check the corresponding boxes to view the number of averages per k-line and cardiac frame, the k-space, or the coil sensitivity maps (ESPIRiT).



Press Start to start the movie.

Press Stop to stop the movie.

Check the corresponding boxes to loop the movie over frames, slices, and/or dynamics.

Image intensity can be adjusted with the level (**L**) and window (**W**) sliders.

Press ↻ or ↺ to rotate the movie clockwise or counter clockwise (this will not affect the Dicom and MRD export orientations).

Press +1 or -1 below **frame** to advance or rewind the movie 1 time frame.

Different slices can be viewed with +1 or -1 below **slice**.

Advance dynamics with +1 or -1 below **dynamic**.

The navigator button Nav toggles the visualization of the navigator slice (if visible in the current slice orientation)

Synchronizing multi-slice 2D CINEs from one animal

Since the cardiac synchronization is not based on ECG triggering, CINEs from different slices or acquired in different orientations may not be synchronized with respect to each other, i.e. the end-systole and end-diastole might have different frame numbers.

To correct for this the following procedure can be followed. First stop the movie with the Stop button. Then set the frame number to 1 in the dialog frame 1 . Then, advance or rewind the movie with +1 or -1 to the desired starting phase of the cardiac cycle, for example end-diastole.

The cardiac synchronization will also be taken into account with DCM, GIF and MRD export.

Step 8: LV segmentation

Press Segment LV to outline the left ventricle in the reconstructed short-axis CINE images and calculate the ventricular volumes.

The segmentation uses a UNet3+ neural network, fine-tuned on rodent cardiac CINE images. It labels every pixel of every frame and every slice as one of three classes:

```
0    background
1    left ventricular myocardium
2    left ventricular blood pool
```

Requirements

Images must be reconstructed first.
The Python environment described in section 5.
macOS, or Windows with WSL.

Each dynamic is treated as a cardiac cycle of its own and is segmented separately. The network was trained on images with a fixed pixel size of 0.13672 mm on a 256 x 256 matrix, which corresponds to a field of view of 35 mm. The app resamples your images onto that grid automatically, so the matrix size and field of view of the acquisition do not matter in themselves. What does matter is that a field of view larger than 35 mm is cropped to the centre. If the heart is not near the centre of the image, part of it can be cut off. The message box reports the field of view against the 35 mm window and warns when it is exceeded.

Segmentation

Before starting the segmentation, make sure that all slices are cardiac phase aligned. See section "Synchronizing multi-slice 2D CINEs from one animal" on previous page.

To start the segmentation, press the 

button.

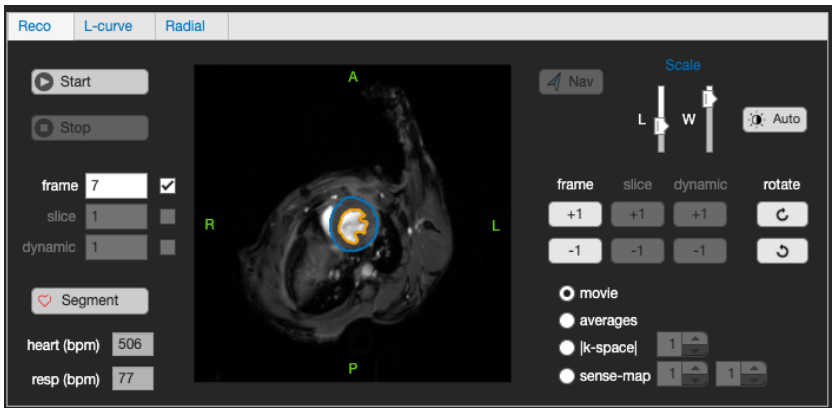
Anatomical clean up

The raw network output is cleaned up with two rules based on anatomy:

(1) The left ventricle is a single connected structure. Where a slice contains more than one separate object, only the one closest to the axis of the ventricle is kept.
(2) The left ventricle lies on a consistent axis through the stack of slices. Where everything found in a slice lies too far from that axis, the ventricle has not been found and the slice is emptied. This is mostly what happens at the base and the apex.

The number of slices that were cleaned or emptied is reported in the message box and is written to the results file, so it can be checked afterwards.

Results



The results of the segmentation are shown in the Reco movie panel. Endocardium and epicardium will be shown as orange and blue contours, respectively.

The following are calculated per dynamic and shown in the message box:

```
EDV         end diastolic volume (ul)
ESV         end systolic volume (ul)
SV          stroke volume (ul)
EF          ejection fraction (%)
ED mass     left ventricular mass at end diastole (mg)
```

Where the slices do not touch, the slice separation rather than the slice thickness is used for the volume, so that the gaps are included. The myocardial mass is calculated with a density of 1.05 g/mL. The contours are drawn over the CINE movie: the epicardium is the outer border of the myocardium and the endocardium the border of the blood pool.
Settings

Segmentation output files

The segmentation results are written together with the exported images, so that the numbers and the contours travel with the data.

(1) **Volumetry**

LVsegmentation_<tag>.xlsx

An Excel file with two sheets.

Summary — one row per dynamic, with the columns:

```
DateTime, Tag, Dynamic, EDframe, ESframe, Frames, Slices,
EDV_ul, ESV_ul, SV_ul, EF_pct, EDmass_mg,
SliceSpacing_mm, ModelPixelSpacing_mm,
CleanUp, DeblobbedSlices, ClearedSlices
```

PerFrame — the blood pool and myocardial volume of every frame:

```
Dynamic, Frame, BloodPool_ul, Myocardium_ul, IsED, IsES
```

This is the curve from which the ED and ES frames were chosen.

(2) **Contours for Medis qMASS**

<PatientID>_STD<StudyID>_SER<n>_ACQ1_ITORIGINAL.con

One file per dynamic, with `_dyn<number>` added to the name when there is more than one. The file contains the endocardial and epicardial contours of every frame and every slice, and can be loaded onto the exported DICOM series.

(3) **ROIs for Horos / OsiriX**

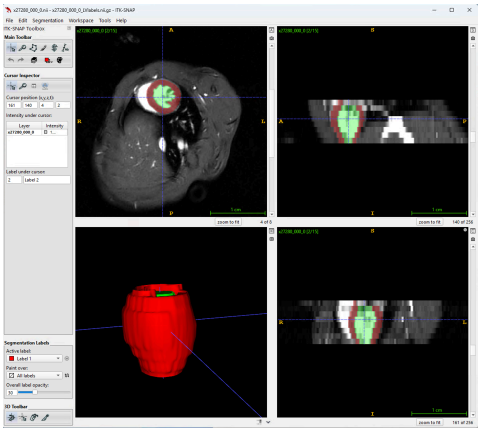
<PatientID>_STD<StudyID>_SER<n>_ACQ1_ITORIGINAL.rois_series

The same contours in the format Horos uses for its own saved ROIs, written next to the exported DICOM files.

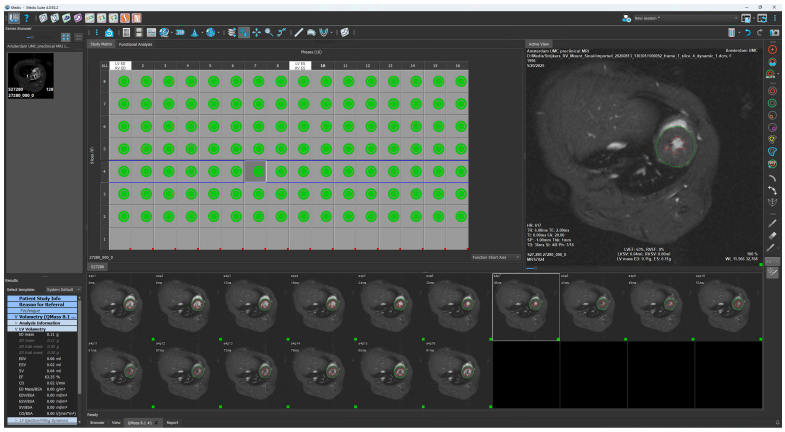
(4) **Label maps as NIfTI**

<name of the CINE file>_LVlabels.nii

Written next to the exported NIfTI CINE, with the same header, so that it can be opened in ITK-SNAP as a segmentation of that image. The label values are the three classes listed at the start of this step.



ITK-SNAP



MEDIS

Step 9: Movie export

There are several ways to export the movies.

(1) **Export DCM**

Exports the data in Dicom and Nifti format for further processing in 3rd party software. The app searches for the Dicom information. If this information is not found, tags will be generated by the app itself. In the latter case the correct image position and orientation information are lost. Dicoms will be saved to /smis/dev/data if found.

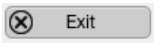
(2) **Export GIF**

Generates animated-gif movie or movies (3D, multi-slice) of the reconstructed CINE. Image intensity (window, level) and image rotations are preserved. Gifs will be saved to /smis/dev/data if found.

(3) **Export MRD**

The data will be stored as a new MRD data file in the same directory as the input MRD file. If the app finds an input rpr file, also a new rpr file will be generated. Additionally, when the app runs on the scanner computer, data will be reconstructed by the scanner software and new SUR and Dicom files will be placed in the data directory for viewing and planning of next scans in the MR Solutions preclinical software.

Step 10: Exit

Press 

to shut down the Retrospective app.

Credits

- BART toolbox and compressed sensing algorithms

  https://mrirecon.github.io/bart/

  Michael Lustig, David Donoho, John M. Pauly, Sparse MRI: The application of compressed sensing for rapid MR imaging, Magn Reson Med 58, 1182-1195 (2007).

- Automatic gradient delay estimation algorithm

  Rosenzweig, S., Holme, H. C. M. & Uecker, M., Simple auto-calibrated gradient delay estimation from few spokes using Radial Intersections (RING). Magn Reson Med 81, 1898–1906 (2019)

- Gibbs ringing suppression

  Kellner, E, Dhital B., Kiselev VG and Reisert, M., Gibbs-ringing artifact removal based on local subvoxel-shifts. Magn Reson Med, 76(5), 1574-1581.

- NUFFT

  https://web.eecs.umich.edu/~fessler/

- Phase unwrapping

  M. A. Herráez, D. R. Burton, M. J. Lalor, and M. A. Gdeisat, "Fast two-dimensional phase-unwrapping algorithm based on sorting by reliability following a noncontinuous path", Applied Optics, Vol. 41, Issue 35, pp. 7437-7444 (2002),

  H. Abdul-Rahman, M. Gdeisat, D. Burton, M. Lalor, "Fast three-dimensional phase-unwrapping algorithm based on sorting by reliability following a non-continuous path", Proc. SPIE 5856, Optical Measurement Systems for Industrial Inspection IV, 32 (2005).

- Frame sharing

  Küstner et al., NMR Biomed e4409 (2020). doi:10.1002/nbm.4409

- Segmentation

  Shah W, Stuckey DJ, Yao T, Wrobel M, Deniszczyc E, Feng Z, Anderson S, Campbell R, Muthurangu V, Steeden J. Open-source pre-clinical image segmentation: mouse cardiac magnetic resonance imaging datasets with a deep learning segmentation framework. J Cardiovasc Magn Reson. 2026 Summer;28(1):102706. doi: 10.1016/j.jocmr.2026.102706. Epub 2026 Feb 17. PMID: 41713653; PMCID: PMC13237537.